



Agentic Controller

Running agent workloads on Kubernetes

September 30, 2026

How this goes

- The problem we're trying to solve
- What we built: architecture and Kubernetes objects
- How Konveyor uses it: Ian's demo
- What's next, then questions

The problem

We want teams to use agents to **modernize applications at scale.**

Define an agent once, then run it across an application inventory.

The people running it need to be able to follow its progress, review its work, and use it without having built the agent themselves.

The landscape

- **Agent Sandbox** provides sandbox lifecycle and isolation. We build on it.
- **OpenShell** adds policy enforcement and model access. Integration is our next step.
- **Lightspeed agentic operator** focuses on cluster remediation.
- **Kagenti** provides security and networking middleware for deployed agents.
- **Fullsend** runs autonomous development workflows on GitHub Actions.

We needed reusable agents and workflows that teams could run across an application inventory.

What we built

Define the building blocks once, then reuse them across runs:

- **Gateway** configures one LLM provider and model: endpoint, credentials, and context window.
- **SkillCard** supplies one skill or rule. Skills load on demand; rules are included in the prompt.
- **SkillCollection** groups related skills so an Agent can reference them together.
- **Agent** combines an image and toolchain, prompt, skills, permitted Gateways, and default execution limits.
- **AgentWorkflow** combines Agents into ordered stages, with a shared guide and instructions for each stage.

Multi-stage workflows

AgentWorkflow defines the stages. Each stage references its own **Agent**.

Konveyor's `java-ee-to-quarkus` workflow:

Stage	Agent	What it does
plan	migration-plan-agent	Reads analysis and writes a migration plan
execute	migration-execute-agent	Applies the plan and records a handoff
verify	migration-verify-agent	Builds, fixes compilation errors, and runs tests

- Each stage can use a **different model**, skills, and execution budget.
- Each AgentRun gets a **fresh Sandbox**; pushed commits on a shared branch carry work to the next stage.

What we built

Create a run to execute an Agent or workflow:

- **AgentRun** invokes one Agent.
 - Supplies parameters, instructions, a Gateway selection, and execution settings.
 - Creates one Sandbox and reports progress and outcome in Kubernetes status.
- **AgentWorkflowRun** invokes an AgentWorkflow.
 - Creates an AgentRun for each stage, in order.
 - Advances after success; stops on failure or an execution limit.

The **Agent / AgentRun** split follows Tekton's **Task / TaskRun** model.

Available in **Konveyor 0.11**, behind `agentic_enabled`.

Design constraints

Our CRDs are the building blocks. These choices shape how people use them:

- **Keep the controller generic.** It reconciles runs and Sandboxes; application details belong in the harness.
- **Make runs observable.** Kubernetes holds status; the UI connects directly to the agent pod for live interaction.
- **Control execution.** The harness and runtime enforce budgets and approval settings.
- **Build on Agent Sandbox.** Each workflow stage gets a fresh sandbox; committed files carry work forward.

The Konveyor harness

- Fetches application details and analysis results from Hub
- Clones the repository and prepares the workspace
- Keeps git push credentials out of the agent's environment and git config; the agent commits locally, and the harness pushes
- Passes work between stages through committed files on a shared branch

A first pass at credential isolation, informed by **Fullsend**.

OpenShell is the next step toward stronger enforcement.



Demo

java-ee-to-quarkus against coolstore

lan

Next: OpenShell

We're targeting **0.12** for an OpenShell execution backend, behind a feature flag.

- Add policy enforcement and stronger credential isolation
- Replace our interim Gateway CRD with OpenShell Gateway Services
- Preserve the Agent / AgentRun model as the backend changes

[OpenShell integration · #150](#)

Where we're headed

Konveyor helps teams modernize their applications.

A migration is part of that work. We want to help teams decide what needs to change, carry out the work, and leave the application easier to maintain and develop.

That includes preserving the decisions and knowledge gained along the way, so the team can keep building on the result.

Questions?

Thanks to everyone who helped build our first release.



Fabian von Feilitzsch · [@fabianvf](#)



Ian Bolton · [@ibolton336](#)



Savitha Raghunathan ·
[@savitharaghunathan](#)



Hiteshwari Patel · [@hhpatel14](#)



Dylan Murray · [@dymurray](#)



Juan Manuel Leflet Estrada · [@jmle](#)



David Zager · [@djzager](#)



Aaron Mark · [@amarkdotdev](#)



[@pttydou](#)